

Method 2: Coupled Muon 的中文推导、实现与测试说明

1 算法详细推导过程

我们考虑两个矩阵参数

$$A \in \mathbb{R}^{m \times r}, \quad B \in \mathbb{R}^{r \times n},$$

它们真正影响模型的是乘积

$$P = AB \in \mathbb{R}^{m \times n}.$$

如果当前的 momentum 梯度分别是

$$M_A \in \mathbb{R}^{m \times r}, \quad M_B \in \mathbb{R}^{r \times n},$$

那么按照一阶近似, 乘积 AB 的变化是

$$\Delta(AB) = AM_B + M_AB.$$

所以我们定义整体的乘积空间 momentum:

$$C = AM_B + M_AB.$$

普通 Muon 是分别处理 M_A 和 M_B 。Method 2 的思想是:

先处理真正影响模型的整体方向 C , 再把它拆回 A 和 B 。

对 C 做 Muon, 得到乘积空间里的目标更新:

$$Z = \text{msign}(C).$$

如果使用 SVD, 设

$$C = U\Sigma V^\top,$$

则

$$Z = U_{[:,1:q]} V_{[:,1:q]}^\top, \quad q = \text{rank}(C).$$

训练时不应该真的用 SVD, 而应该和 Muon 一样使用 Newton-Schulz 近似:

$$X_0 = \frac{C}{\|C\|_F + \epsilon},$$

$$X_{t+1} = 3.4445X_t - 4.7750(X_t X_t^\top)X_t + 2.0315(X_t X_t^\top)^2 X_t.$$

现在我们要找到两个真正用于更新参数的矩阵

$$U_A \in \mathbb{R}^{m \times r}, \quad U_B \in \mathbb{R}^{r \times n},$$

使得

$$AU_B + U_AB \approx Z.$$

这里有一个关键问题：并不是任意的 Z 都能写成 $AU_B + U_AB$ 。所有能写出来的矩阵组成一个 tangent space:

$$\mathcal{T}_{A,B} = \{AD_B + D_AB\}.$$

所以正确的目标不是强行等于 Z ，而是先把 Z 投影到可达空间：

$$Z_T = \Pi_{\mathcal{T}_{A,B}}(Z).$$

令 A^+ 和 B^+ 表示 Moore-Penrose 伪逆，定义

$$P_A = AA^+, \quad P_B = B^+B.$$

其中 P_A 是列空间投影， P_B 是行空间投影。可达空间投影公式为

$$Z_T = P_A Z + Z P_B - P_A Z P_B.$$

之后我们只需要满足

$$AU_B + U_AB = Z_T.$$

一种显式可行的拆分方式是：

$$R = C - Z_T,$$

$$U_B = M_B - A^+ R,$$

$$U_A = M_A - (I - P_A) R B^+.$$

验证如下：

$$AU_B + U_AB = A(M_B - A^+ R) + (M_A - (I - P_A) R B^+) B.$$

展开：

$$AU_B + U_AB = AM_B + M_AB - AA^+ R - (I - P_A) R B^+ B.$$

因为

$$C = AM_B + M_AB, \quad P_A = AA^+, \quad P_B = B^+ B,$$

所以

$$AU_B + U_AB = C - P_A R - (I - P_A) R P_B.$$

又因为 $R = C - Z_T$ 本身也在可达空间 $\mathcal{T}_{A,B}$ 中，所以

$$R = P_A R + (I - P_A) R P_B.$$

代入可得

$$AU_B + U_AB = C - R = Z_T.$$

所以这个拆分是正确的。总结一句话：

Method 2 可以精确实现投影后的目标 Z_T ，但一般不能保证精确实现原始目标 Z 。

2 算法的计算实现方法

2.1 通用版本

通用版本适用于任意

$$A \in \mathbb{R}^{m \times r}, \quad B \in \mathbb{R}^{r \times n}.$$

计算步骤如下:

1. 更新 momentum:

$$M_A \leftarrow \beta M_A + G_A, \quad M_B \leftarrow \beta M_B + G_B.$$

2. 构造乘积空间方向:

$$C = AM_B + M_AB.$$

3. 对 C 做 Muon:

$$Z = \text{msign}(C).$$

4. 计算伪逆和投影矩阵:

$$A^+, \quad B^+, \quad P_A = AA^+, \quad P_B = B^+B.$$

5. 投影到可达空间:

$$Z_T = P_A Z + Z P_B - P_A Z P_B.$$

6. 拆回两个参数更新:

$$R = C - Z_T,$$

$$U_B = M_B - A^+ R,$$

$$U_A = M_A - (I - P_A) R B^+.$$

7. 更新参数:

$$A \leftarrow A - \eta_A U_A, \quad B \leftarrow B - \eta_B U_B.$$

通用版本数学上最完整, 但计算很贵。尤其不要在大模型训练中显式构造很大的 P_A 和 P_B , 否则内存和时间都会很差。

2.2 LLaMA q/k 的特化版本

在 HuggingFace 的 LlamaForCausalLM 模型中。对于 q/k 矩阵:

$$W_q, W_k \in \mathbb{R}^{h \times d},$$

其中

$$d = \text{hidden_size}, \quad h = \text{num_heads} \times \text{head_dim}.$$

attention score 中自然出现的乘积是

$$W_q^\top W_k.$$

所以在测试脚本中我们取

$$A = W_q^\top \in \mathbb{R}^{d \times h}, \quad B = W_k \in \mathbb{R}^{h \times d}.$$

对于当前配置，通常 $h \geq d$ ，随机初始化下 A 几乎一定满行秩。因此

$$AA^+ = I_d.$$

这意味着任意

$$Z \in \mathbb{R}^{d \times d}$$

都可以写成

$$AU_B + U_AB.$$

也就是说 q/k 这个特化场景下：

$$Z_T = Z.$$

算法可以简化成：

$$C = AM_B + M_AB,$$

$$Z = \text{msign}(C),$$

$$R = C - Z,$$

$$U_A = M_A,$$

$$U_B = M_B - A^+R.$$

实现时不要显式求逆。若 A 是方阵，直接解线性方程：

$$AX = R,$$

然后

$$U_B = M_B - X.$$

若 A 是宽矩阵并满行秩，用

$$A^+R = A^\top(AA^\top)^{-1}R.$$

实现时写成两步：

$$(AA^\top)Y = R,$$

$$A^+R = A^\top Y.$$

这样比显式计算

$$(AA^\top)^{-1}$$

更稳定。

3 算法的精度分析和运行效率分析结果

3.1 精度分析

我们主要检查这个相对误差：

$$E_T = \frac{\|AU_B + U_AB - Z_T\|_F}{\|Z_T\|_F}.$$

如果算法正确， E_T 应该接近机器精度。

在 `test_method2_sanity.py` 中，使用随机矩阵测试了三种情况：

case	rank(A)	rank(B)	E_T
full-rank thin factor	3	3	3.694×10^{-15}
full-rank wider factor	5	6	1.943×10^{-15}
rank-deficient factor	3	3	1.308×10^{-15}

这些结果说明：只要目标用 Z_T ，拆分公式可以非常精确地满足

$$AU_B + U_AB = Z_T.$$

同一个测试还说明，直接追原始 Z 不一定可行。例如：

$$\frac{\|AU_B + U_AB - Z\|_F}{\|Z\|_F} \approx 3.4 \times 10^{-1}$$

在某些随机低秩 case 中明显不小。所以投影步骤不是多余的，而是保证算法正确的关键。

3.2 q/k 初始化下的精度

在 `benchmark_method2_llama_qk.py` 中，我们按照 `model_llama.py` 里的 q/k 形状进行随机初始化：

$$W_q, W_k \sim \mathcal{N}(0, 0.02^2),$$

并用随机 momentum gradient 测试。

对于 93M 配置：

$$d = 512, \quad h = 16 \times 64 = 1024.$$

因此

$$A = W_q^\top \in \mathbb{R}^{512 \times 1024}, \quad B = W_k \in \mathbb{R}^{1024 \times 512}.$$

测试结果：

$$E_T = 1.130 \times 10^{-15}.$$

对于 130M 配置：

$$d = 768, \quad h = 12 \times 64 = 768.$$

测试结果：

$$E_T = 1.188 \times 10^{-11}.$$

这个误差略大于 93M，是因为这里 A 是 768×768 方阵，线性求解的数值条件更敏感；但 10^{-11} 仍然是非常小的数值误差。

3.3 运行效率分析

设 Newton–Schulz 迭代次数为 K 。

普通 Muon 分别处理 M_A 和 M_B ，复杂度大约是

$$O(K \min(m, r)^2 \max(m, r) + K \min(r, n)^2 \max(r, n)).$$

通用 Method 2 需要先形成

$$C = AM_B + M_AB,$$

复杂度为

$$O(mrn).$$

然后还要对

$$C \in \mathbb{R}^{m \times n}$$

做 Muon，复杂度大约是

$$O(K \min(m, n)^2 \max(m, n)).$$

如果再加上伪逆、投影矩阵和 Z_T 的构造，通用 Method 2 会比普通 Muon 贵很多。实际测试中，93M q/k 形状下的通用投影版本大约是普通 Muon 的

$$5.10 \times$$

耗时。

3.4 q/k 特化版本的效率

q/k 特化版本利用了

$$A = W_q^\top$$

通常满行秩，因此

$$Z_T = Z.$$

这样可以跳过通用投影，只需要：

$$C = AM_B + M_AB,$$

$$Z = \text{msign}(C),$$

$$AX = C - Z,$$

$$U_B = M_B - X, \quad U_A = M_A.$$

实际 NumPy CPU 测试结果如下：

model	普通 Muon best	Method 2 best	比值
93M	0.220053s	0.192228s	$0.87 \times$
130M	0.306900s	0.342304s	$1.12 \times$
93M generic	0.220843s	1.126366s	$5.10 \times$

注意：这些是 NumPy CPU 测试，不等于最终 PyTorch GPU 训练速度，只能说明趋势。结论是：

- 通用 Method 2 数学正确，但很贵，不建议直接用于大矩阵训练。
- q/k 特化版本因为 $A = W_q^\top$ 通常满行秩，可以把 $Z_T = Z$ ，因此速度可以接近普通 Muon。
- 在 93M q/k 形状下，特化 Method 2 甚至略快于普通 Muon；在 130M 下略慢。
- 通用投影版本明显慢，当前测试约为普通 Muon 的 $5.10\times$ 。
- 如果未来真正接入训练循环，应该优先实现 q/k 特化版本，而不是通用版本。

4 最终建议

我建议把 Method 2 分成两个层级：

1. 研究版通用算法：保留 Z_T 投影公式，用于理解和小矩阵验证。
2. 训练版 q/k 特化算法：利用 $A = W_q^\top$ 满行秩，直接令 $Z_T = Z$ ，用线性方程求解拆分。

如果要接入当前工程，下一步最合理的是：

先只在 q/k pair 上实现 Method 2 fast split，并和普通 Muon、Method 1 做小模型训练对比。

5 训练实现中的问题与 Balanced Split 修正

上面的 q/k 特化版本最初使用的是一种单边拆分：

$$C = Q^\top M_K + M_Q^\top K, \quad Z = \text{msign}(C), \quad R = C - Z,$$

$$U_Q = M_Q, \quad U_K = M_K - Q(Q^\top Q + \epsilon I)^{-1}R.$$

这个方案在数学上看起来很直接：把所有 correction 都放到 K 上，使得

$$Q^\top U_K + U_Q^\top K \approx Z.$$

但是训练中它有一个严重问题：真实写回参数的是 U_Q, U_K ，而不是乘积空间里的 Z 。普通 Muon 会直接把每个参数矩阵的 momentum 正交化，所以每个参数的更新范数基本稳定；单边拆分只控制了 $Q^\top U_K + U_Q^\top K$ ，没有控制 U_K 本身的范数。

更具体地说，单边拆分包含

$$Q(Q^\top Q + \epsilon I)^{-1}R.$$

如果 Q 出现小奇异值，那么这个项会沿对应方向接近似

$$\frac{\sigma_Q}{\sigma_Q^2 + \epsilon}$$

放大。随着训练推进, $Q^\top Q$ 的条件数可能变差, 于是 $\|U_K\|_F$ 会持续上升, 进一步破坏 q/k 的几何结构, 形成正反馈。这解释了实际日志中 `update_norm/qk` 随 `step` 增加而指数上升的现象。

另一个问题是 multihead 结构。旧实现虽然会把 GQA 中的 K head repeat 到 Q head 数量, 但之后仍然把所有 head 堆成一个大矩阵:

$$Q = \begin{bmatrix} Q_1 \\ Q_2 \\ \vdots \\ Q_H \end{bmatrix}, \quad K = \begin{bmatrix} K_1 \\ K_2 \\ \vdots \\ K_H \end{bmatrix}.$$

这样得到的目标是

$$Q^\top K = \sum_{h=1}^H Q_h^\top K_h.$$

也就是说, 旧实现优化的是所有 head product 的总和, 然后只做一次 `msign`。但是 attention score 是按 head 分开计算的; 更自然的目标应该是每个 head 独立处理:

$$C_h = Q_h^\top M_{K,h} + M_{Q,h}^\top K_h, \quad Z_h = \text{msign}(C_h).$$

注意一般有

$$\text{msign}\left(\sum_h C_h\right) \neq \sum_h \text{msign}(C_h).$$

如果不同 head 的方向互相抵消, 全局版本会在进入 Muon 之前就丢掉 head 级别的信息。

新的训练版实现因此改成 per-head balanced split。对每个 head h , 把 correction 同时分配给 Q_h 和 K_h :

$$U_{Q,h} = M_{Q,h} - K_h \Lambda_h^\top, \quad U_{K,h} = M_{K,h} - Q_h \Lambda_h.$$

代入乘积空间更新得到

$$Q_h^\top U_{K,h} + U_{Q,h}^\top K_h = C_h - Q_h^\top Q_h \Lambda_h - \Lambda_h K_h^\top K_h.$$

因此 exact balanced split 只需要解 Sylvester 方程:

$$Q_h^\top Q_h \Lambda_h + \Lambda_h K_h^\top K_h = C_h - Z_h.$$

为了训练稳定, 实际实现使用 damped/proximal 版本:

$$Q_h^\top Q_h \Lambda_h + \Lambda_h K_h^\top K_h + \lambda \Lambda_h = C_h - Z_h,$$

其中

$$\lambda = \rho \cdot \frac{\text{mean diag}(Q_h^\top Q_h) + \text{mean diag}(K_h^\top K_h)}{2}.$$

当前代码中默认使用 $\rho = 0.1$ 。如果令

$$Q_h^\top Q_h = S_Q D_Q S_Q^\top, \quad K_h^\top K_h = S_K D_K S_K^\top,$$

则可以用特征分解高效求解：

$$\Lambda_h = S_Q \left(\frac{S_Q^\top (C_h - Z_h) S_K}{D_Q \mathbf{1}^\top + \mathbf{1} D_K^\top + \lambda} \right) S_K^\top.$$

这个版本的关键差别是，旧方法的放大分母主要来自 $\sigma_Q^2 + \epsilon$ ，而 balanced split 的分母来自

$$\sigma_{Q,i}^2 + \sigma_{K,j}^2 + \lambda.$$

只要 Q 和 K 不在同一个方向同时退化，放大就会小很多；即便两者都退化， λ 也会提供下界。

需要注意的是，当 $\lambda > 0$ 时，乘积空间目标不再被精确满足：

$$Q_h^\top U_{K,h} + U_{Q,h}^\top K_h = Z_h + \lambda \Lambda_h.$$

这是有意的折中：放弃一点 product-space 精确度，换取参数空间更新范数稳定。

旧的训练实现曾经尝试对最终写回参数的两个矩阵分别做一次 Muon orthogonalization,

$$\tilde{U}_Q = \text{msign}(U_Q), \quad \tilde{U}_K = \text{msign}(U_K).$$

这个做法虽然能让 `update_norm/qk` 平稳，但会破坏刚刚通过 Sylvester 方程解出的 product-space 方向。新的实现不再分别 `msign`(U_Q) 和 `msign`(U_K)，而是只做一个共同的 scalar normalization:

$$s = \frac{\sqrt{\min(\text{shape}(U_Q)) + \min(\text{shape}(U_K))}}{\sqrt{\|U_Q\|_F^2 + \|U_K\|_F^2 + \epsilon}},$$

$$\tilde{U}_Q = s U_Q, \quad \tilde{U}_K = s U_K.$$

这样最终乘积空间更新只是原始 raw split 方向的整体缩放：

$$Q^\top \tilde{U}_K + \tilde{U}_Q^\top K = s(Q^\top U_K + U_Q^\top K),$$

不会像 final Muon 那样改变方向本身。它不是 norm cap；它不会在超过阈值时截断更新，而是始终用同一个标量把 q/k 的 combined update 调到 Muon 量级。

对于 GQA，一个 K head 会服务多个 Q head。实现中先把 K head repeat 到 Q head 数量，逐 head 得到 $U_{K,h}$ ，然后把属于同一个原始 K head 的多个更新平均回去。

新的训练版 Method 2 不再追求精确实现 Z ，而是追求 per-head Z_h 的稳定近似：per-head balanced correction + damped Sylvester solve + joint scalar normalization。

6 算法演变过程整理

本节记录 Method 2 / Tangent Coupled Muon 从最初推导到当前实验版本的演变。这样后续继续试算法时，可以清楚地区分：

- 哪些问题已经被实验排除；
- 哪些改动只是加速实现，不改变算法目标；
- 哪些改动真正改变了优化器的 product-space target。

6.1 统一记号

对一个 q/k head, 记

$$Q, K, M_Q, M_K \in \mathbb{R}^{d_h \times d},$$

其中 d_h 是 head dimension, d 是 hidden size。定义线性映射

$$\mathcal{L}_{Q,K}(U_Q, U_K) = Q^\top U_K + U_Q^\top K.$$

所有 Tangent Coupled Muon 版本本质上都分成两步:

1. 从 momentum 构造某个 product-space target;
2. 找到参数空间更新 U_Q, U_K , 使

$$\mathcal{L}_{Q,K}(U_Q, U_K)$$

尽量接近这个 target。

不同版本的区别主要在于:

- target 是在什么空间里做 msign;
- split 是否保留 momentum 中对 product 无作用的 kernel 分量;
- 是否显式考虑 per-head、RoPE、低秩结构。

6.2 V0: 通用 Method 2

最初的数学目标是直接在乘积

$$P = AB$$

上做 Muon。给定 momentum

$$M_A, \quad M_B,$$

先构造

$$C = AM_B + M_AB,$$

再做

$$Z = \text{msign}(C).$$

由于 Z 不一定属于 tangent space

$$\mathcal{T}_{A,B} = \{AD_B + D_AB\},$$

理论上需要先投影:

$$Z_T = \Pi_{\mathcal{T}_{A,B}}(Z).$$

这个版本数学最完整,但训练中不可接受,因为需要大矩阵 product、投影、伪逆或大 Sylvester solve。它主要作为研究版和 sanity check。

6.3 V1: q/k fast one-sided split

对 q/k, attention 中自然出现

$$W_q^\top W_k.$$

如果令

$$A = W_q^\top, \quad B = W_k,$$

在全局 q/k 形状下, A 通常满行秩, 于是有

$$Z_T = Z.$$

因此可以使用单边拆分:

$$C = Q^\top M_K + M_Q^\top K, \quad Z = \text{msign}(C),$$

$$R = C - Z,$$

$$U_Q = M_Q, \quad U_K = M_K - Q(Q^\top Q + \epsilon I)^{-1} R.$$

这个版本计算简单, 但问题很明显: 所有 correction 都压到 K 上。当 $Q^\top Q$ 条件数变差时, U_K 会被小奇异方向放大, 导致 q/k update norm 随训练推进变大。这个版本被稳定性问题淘汰。

6.4 V2: per-head balanced split

后来改成 per-head target:

$$C_h = Q_h^\top M_{K,h} + M_{Q,h}^\top K_h, \quad Z_h = \text{msign}(C_h).$$

再用 balanced split:

$$U_{Q,h} = M_{Q,h} - K_h \Lambda_h^\top, \quad U_{K,h} = M_{K,h} - Q_h \Lambda_h.$$

代入可得:

$$\mathcal{L}_{Q_h, K_h}(U_{Q,h}, U_{K,h}) = C_h - Q_h^\top Q_h \Lambda_h - \Lambda_h K_h^\top K_h.$$

因此解

$$Q_h^\top Q_h \Lambda_h + \Lambda_h K_h^\top K_h + \lambda \Lambda_h = C_h - Z_h.$$

这个版本解决了单边 split 的范数爆炸问题, 也避免了全 head 相加后再 msign 的信息抵消。但是训练中观察到: 最开始 loss 明显优于 Muon 和 Coupled Muon, 随后优势快速缩小, 后期甚至差于 Muon 与 q/k-only Coupled Muon。

这个现象说明: per-head balanced split 虽然更稳定, 但 target

$$Z_h = \text{msign}(C_h)$$

本身可能不是长期有效的优化方向。

6.5 V3: 低秩加速实现

per-head 的

$$C_h = Q_h^\top M_{K,h} + M_{Q,h}^\top K_h$$

天然就是低秩矩阵，因为

$$C_h = \begin{bmatrix} Q_h^\top & M_{Q,h}^\top \end{bmatrix} \begin{bmatrix} M_{K,h} \\ K_h \end{bmatrix}.$$

所以

$$\text{rank}(C_h) \leq 2d_h.$$

于是可以不用对 $d \times d$ 的 dense C_h 做 Newton-Schulz，而是先做 compact QR:

$$L = Q_L L_R, \quad R^\top = Q_R R_R,$$

然后只在小 core

$$L_R R_R^\top$$

上做 NS。

同理，balanced Sylvester 也可以通过低秩因子和小矩阵求解完成，避免 hidden-sized eigen-solve。

实验结果：

- 低秩实现与 dense 当前算法的 U_Q, U_K 数值误差约为 10^{-14} ;
- LLaMA-93M q/k 形状下，NumPy CPU benchmark 约有 $2\times$ 以上加速;
- 训练 loss 曲线与 dense 版本几乎完全相同。

结论：低秩实现是有效的工程优化，但不改变算法行为；后期变差不是由实现误差导致。

6.6 V4: Horizontal / minimum-norm split

接着尝试排除 kernel drift 假设。balanced split 实际上选择的是“离原始 momentum 最近”的可行更新：

$$U = M - \mathcal{L}^*(\mathcal{L}\mathcal{L}^* + \lambda I)^{-1} (\mathcal{L}(M) - Z).$$

这个更新会保留 momentum 中满足

$$\mathcal{L}_{Q,K}(U_Q, U_K) = 0$$

的 kernel/gauge 分量。于是尝试改成 minimum-norm horizontal update:

$$\min_{U_Q, U_K} \|U_Q\|_F^2 + \|U_K\|_F^2 \quad \text{s.t.} \quad Q^\top U_K + U_Q^\top K \approx Z.$$

解的形式为

$$U_Q = K\Lambda^\top, \quad U_K = Q\Lambda,$$

其中

$$Q^\top Q \Lambda + \Lambda K^\top K + \lambda \Lambda = Z.$$

实验结果: loss 曲线与 previous balanced split 几乎一样。

结论: kernel/gauge drift 不是当前 loss 后期落后的主因; 问题更可能在 target $\text{msign}(C)$ 本身。

6.7 V5: RoPE-aware real/cross target

LLaMA attention 在 q/k 上使用 RoPE。若把一个 rotary pair 写成

$$q = (q_1, q_2), \quad k = (k_1, k_2),$$

那么相对位置 Δ 下的 attention score 包含

$$\cos(\Delta\theta)(q_1 k_1 + q_2 k_2) + \sin(\Delta\theta)(q_2 k_1 - q_1 k_2).$$

所以不仅有 ordinary product

$$R = q_1 k_1 + q_2 k_2,$$

还有 cross product

$$S = q_2 k_1 - q_1 k_2.$$

因此尝试把 head 按 rotary bucket 拆开, 对每个 bucket 同时构造两个 target:

$$C_R = Q^\top M_K + M_Q^\top K,$$

$$C_S = Q^\top J M_K + M_Q^\top J K,$$

其中 J 表示 90° 旋转。然后分别做

$$Z_R = \text{msign}(C_R), \quad Z_S = \text{msign}(C_S),$$

再把两个 update 合起来。

实验结果: loss 比 previous 版本略低, 约有 0.003 改善, 但仍然明显差于 Muon 和 Coupled Muon, 约比 Coupled Muon 高 0.012。

结论: RoPE 结构可能有一点帮助, 但 real/cross 分别 msign 不是根本解决方案。

6.8 V6: Complex RoPE target

进一步尝试把 rotary pair 写成复数:

$$q_c = q_1 + iq_2, \quad k_c = k_1 + ik_2.$$

于是 RoPE 相关 product 可写为

$$q_c^H k_c.$$

对应的 tangent momentum 是

$$C_c = Q_c^H M_{K,c} + M_{Q,c}^H K_c.$$

然后在复数 product space 中做

$$Z_c = \text{msign}(C_c),$$

再 split 回复数 $U_{Q,c}, U_{K,c}$, 最后还原成实数更新。

实验结果：效果反而差于 V5。

结论：简单地把 RoPE pair 复数化并做 complex polar/msign 并不合适。可能原因是 complex product target 过于激进，或者与实际 attention loss 的统计权重不匹配。

6.9 V7: Lambda-space Tangent Coupled Muon

目前最后尝试的方向是：不要直接对 product momentum

$$C = Q^\top M_K + M_Q^\top K$$

做 msign。因为 C 已经被 $Q^\top Q$ 和 $K^\top K$ 的几何扭曲。

先把 C 还原到 horizontal coefficient space:

$$Q^\top Q \Lambda_{\text{raw}} + \Lambda_{\text{raw}} K^\top K + \lambda \Lambda_{\text{raw}} = C.$$

然后在 Λ -space 中做 Muon:

$$\Lambda = \text{msign}(\Lambda_{\text{raw}}).$$

最后推回参数空间:

$$U_Q = K \Lambda^\top, \quad U_K = Q \Lambda.$$

这版的动机是： Λ 是真正决定 horizontal update 的系数。如果直接对 C 做 msign，相当于在被当前 Q, K Gram 扭曲后的坐标里正交化；而对 Λ_{raw} 做 msign，更像是在更新系数空间里正交化。

实现中仍然使用低秩因子，避免显式构造 dense Λ_{raw} 。

6.10 目前被实验排除或削弱的假设

- 只是实现太慢或数值不准：低秩实现复现 dense 版本到数值精度，但 loss 不变。
- 只是只作用 q/k、不公平：实验中已有 q/k-only Coupled Muon，排序仍然是

$$\text{Coupled Muon} < \text{Coupled Muon}(q/k \text{ only}) < \text{Muon} < \text{Tangent Coupled Muon}.$$

- 只是 split 保留了 kernel/gauge drift：minimum-norm horizontal split 与 balanced split 的 loss 几乎相同。
- 只是没有 RoPE 结构：real/cross RoPE target 有小幅改善，但 complex RoPE target 变差。

6.11 当前判断

目前最可疑的核心问题仍然是 `msign` 的作用空间和强度:

- 普通 Muon 是对参数 `momentum` 做 `msign`;
- Coupled Muon 是在乘法结构约束下对每个参数方向做类似的 `coupled NS`;
- Tangent Method 2 则试图对 `product-space` 或 `coefficient-space` 方向做 `msign`。

训练早期 Tangent Coupled Muon 很好, 说明 `product-level` 信号确实有效; 但后期迅速被 Muon / Coupled Muon 反超, 说明 `hard msign` 可能放大了 `product-space` 或 `coefficient-space` 中后期噪声较大的小奇异方向。

如果 V7 仍然不能追上 Muon / Coupled Muon, 下一步更合理的算法方向可能不是继续改 `split`, 而是改 `msign` 本身, 例如:

$$U\Sigma V^\top \implies U\phi(\Sigma)V^\top,$$

其中 ϕ 不再把所有非零奇异值都压成 1, 而是使用 `truncated`、`energy-threshold` 或 `soft sign` 的形式。这个方向还没有正式实验。